Lab3:上板运行 实验报告

PB24111621 肖瀚

一、实验目的与内容

1. 了解熟悉 FPGAOL 网站的使用，包括按钮、开关、LED、数码管、串口界面等；
2. 掌握使用 Vivado 开发项目并在 FPGAOL 上运行的流程：从代码设计到综合与实现，再到生成比特流文件在 FPGAOL 上运行；
3. 了解基础的信号处理技术，学习了消除毛刺、边缘检测以及时钟分频等知识；
4. 了解 Verilog 一些常见问题，如位宽不匹配、值不可达等，知道在编写代码时尽量避免相关的错误操作，以及在出现相关的 Warning 时该如何去解决。

二、逻辑设计

【必做内容】

题目1：开关与 LED

根据题目要求，只需要将开关 sw 的值对应赋给 led 即可，代码如下：

```
module Top (
    input   [7:0]                       sw,
    output  [7:0]                       led
);
    assign led[3:0] = {sw[0], sw[1], sw[2], sw[3]};
    assign led[7:4] = {sw[4], sw[5], sw[6], sw[7]};
endmodule
```

题目2：计数器 Pro Plus

经计算，要使LED以 1Hz 的频率闪烁，计数器要每次到达 50_000_000 后置零，并且至少需要位宽为 26 的计数器变量 counter 。每次在达到 `counter == 1` 时变量 led 会进行一次取反操作，保证 LED[7:0] 在亮0.5s 后全部都熄灭 0.5s ，在熄灭 0.5s 后又会亮 0.5s 。实验代码如下：

```
module Counter #(
    parameter   MAX_VALUE = 26'd50_000_000
)(
    input                   clk,
    input                   btn,
    output reg [7:0]        led
);

reg [25:0] counter;
assign rst = btn;

always @(posedge clk) begin
```

```
    if (rst)
        counter <= 0;
    else begin
        if (counter >= MAX_VALUE)
            counter <= 0;
        else
            counter <= counter + 26'b1;
    end
end

always @(posedge clk) begin
    if (rst)
        led <= 8'b0000_0000;
    else if (counter == 1) begin
        led <= ~led;
    end
end

endmodule
```

题目3：七段数码管

- 问题1：计时切换当前被点亮的数码管编号，这里对计数变量 counter 的处理与之前基本一致，在达到最大技术次数后置零，否则自增。而每次在 `counter == 1` 的时候更新一次 seg_id ，因其变量位宽为3，则刚好可以表示 8 个数码管。
- 问题2：根据编号确定数码管显示的内容，将 seg_id 的值赋给 seg_an ，也就是确定数码管的编号，再将对应位置的 output_data 的值赋给 seg_data ，这样便能实现数码管对应位置显示所需的数字。

将上面两点结合起来，便可以得到下面的代码：

```
module Segment(
    input                       clk,
    input                       btn,
    input       [31:0]          output_data,

    output reg  [ 3:0]          seg_data,
    output reg  [ 2:0]          seg_an
);

reg [31:0] counter;
reg [2:0] seg_id;
parameter TIME_CNT = 250_000;

always @(posedge clk) begin
    if (btn)
        counter <= 0;
    else if (counter >= TIME_CNT)
        counter <= 0;
    else
        counter <= counter + 1;
end
```

```
always @(posedge clk) begin
    if (btn)
        seg_id <= 0;
    else if(counter == 1)
        seg_id <= seg_id + 1;
end

always @(*) begin
    seg_data <= 0;
    seg_an <= seg_id;    // <- Same for all cases

    case(seg_id)
        3'b000: seg_data <= output_data[3:0];
        3'b001: seg_data <= output_data[7:4];
        3'b010: seg_data <= output_data[11:8];
        3'b011: seg_data <= output_data[15:12];
        3'b100: seg_data <= output_data[19:16];
        3'b101: seg_data <= output_data[23:20];
        3'b110: seg_data <= output_data[27:24];
        3'b111: seg_data <= output_data[31:28];
    endcase
    // Update seg_data according to seg_id. Hint: Use "case".
end

endmodule
```

【选择性必做内容】

题目3：带有掩码的数码管

这道题目跟第三题没有太大差别，只需要修改 seg_an 信号和 seg_data 信号的产生逻辑，增添一条 if-else 语句来判断：如果该位需要被掩盖，那么可以让对应的 seg_an 变为 3'b000，让 seg_data 变为 output_data[3:0]，即让最低位的数码管保持亮的状态，而不会接通需要掩盖位的数码管，从而实现“掩码”；若不需要被掩盖，则正常赋值。模块代码如下：

```
module Segment2(
    input                       clk,
    input                       btn,
    input       [31:0]          output_data,
    input       [ 7:0]          output_valid,

    output reg  [ 3:0]          seg_data,
    output reg  [ 2:0]          seg_an
);

reg [31:0]  counter;
reg [2:0]   seg_id;
parameter   TIME_CNT = 250_000;
```

```
always @(posedge clk) begin
    if (btn)
        counter <= 0;
    else if (counter >= TIME_CNT)
        counter <= 0;
    else
        counter <= counter + 1;
end

always @(posedge clk) begin
    if (btn)
        seg_id <= 0;
    else if(counter == 1)
        seg_id <= seg_id + 1;
end

always @(*) begin
    seg_an = seg_id;

    if (output_valid[seg_id] != 1'b0)    begin
        case(seg_id)
            3'b000: seg_data = output_data[3:0];
            3'b001: seg_data = output_data[7:4];
            3'b010: seg_data = output_data[11:8];
            3'b011: seg_data = output_data[15:12];
            3'b100: seg_data = output_data[19:16];
            3'b101: seg_data = output_data[23:20];
            3'b110: seg_data = output_data[27:24];
            3'b111: seg_data = output_data[31:28];
        endcase
    end
    else    begin
        seg_an = 3'b000;
        seg_data = output_data[3:0];
    end
end

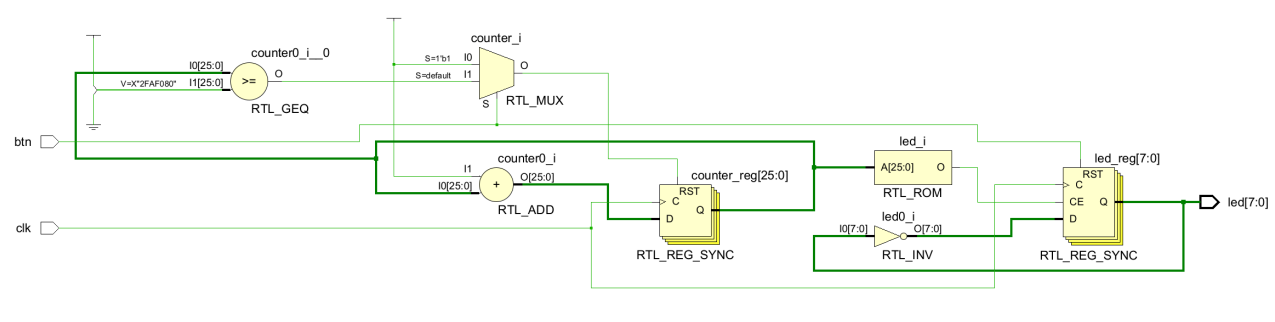
endmodule
```

三、电路设计与分析

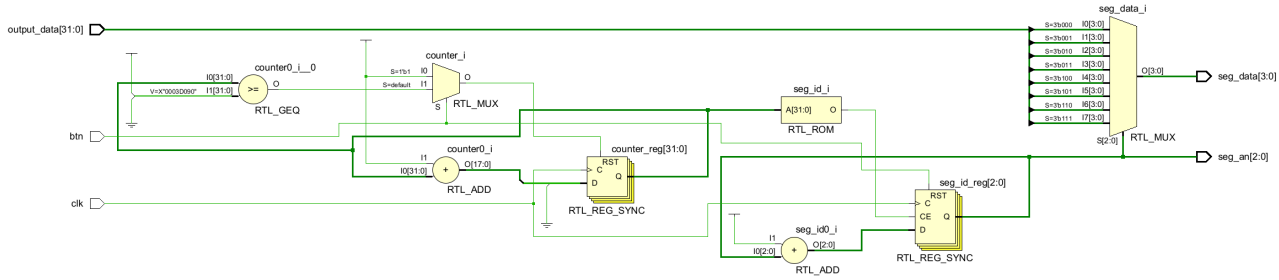
1. 必做题目1的 RTL 电路图如下：



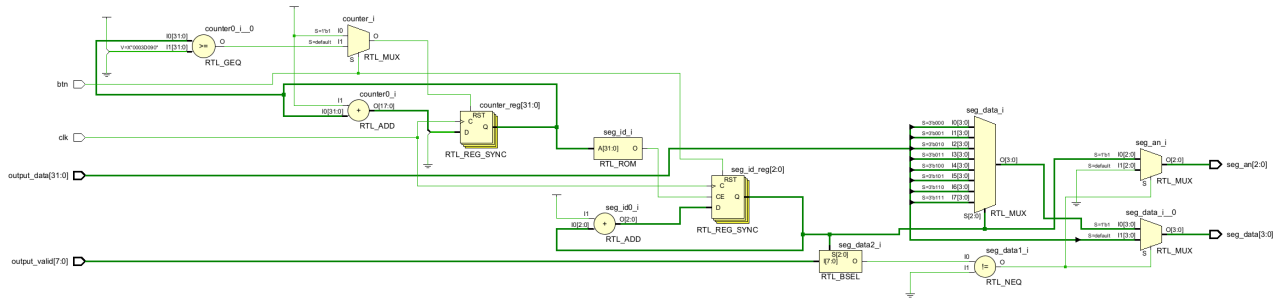
2. 必做题目2的 RTL 电路图如下：



3. 必做题目3的 RTL 电路图如下：

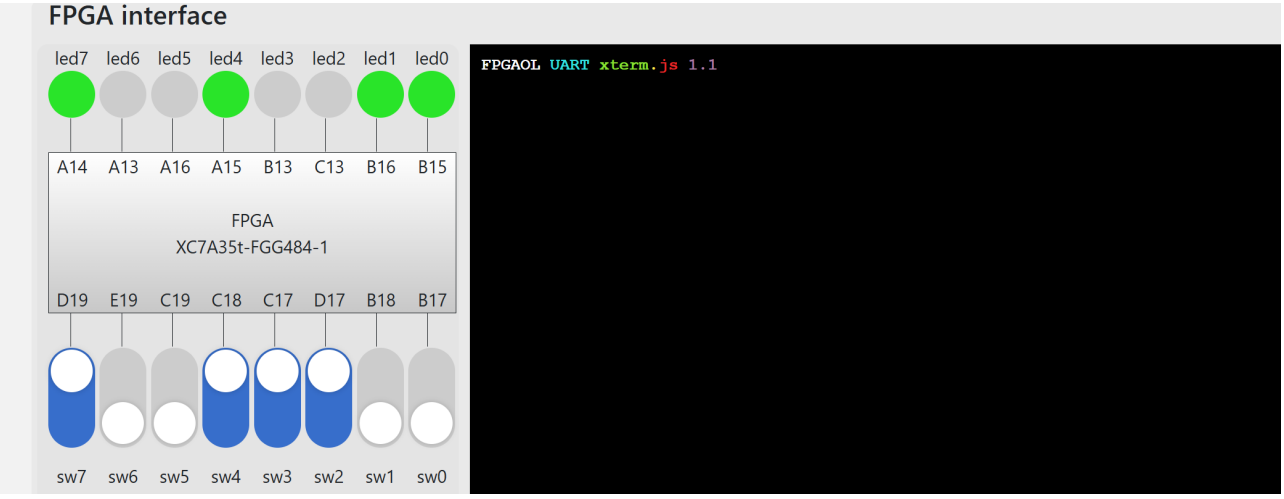


4. 选做题目3的 RTL 电路图如下：

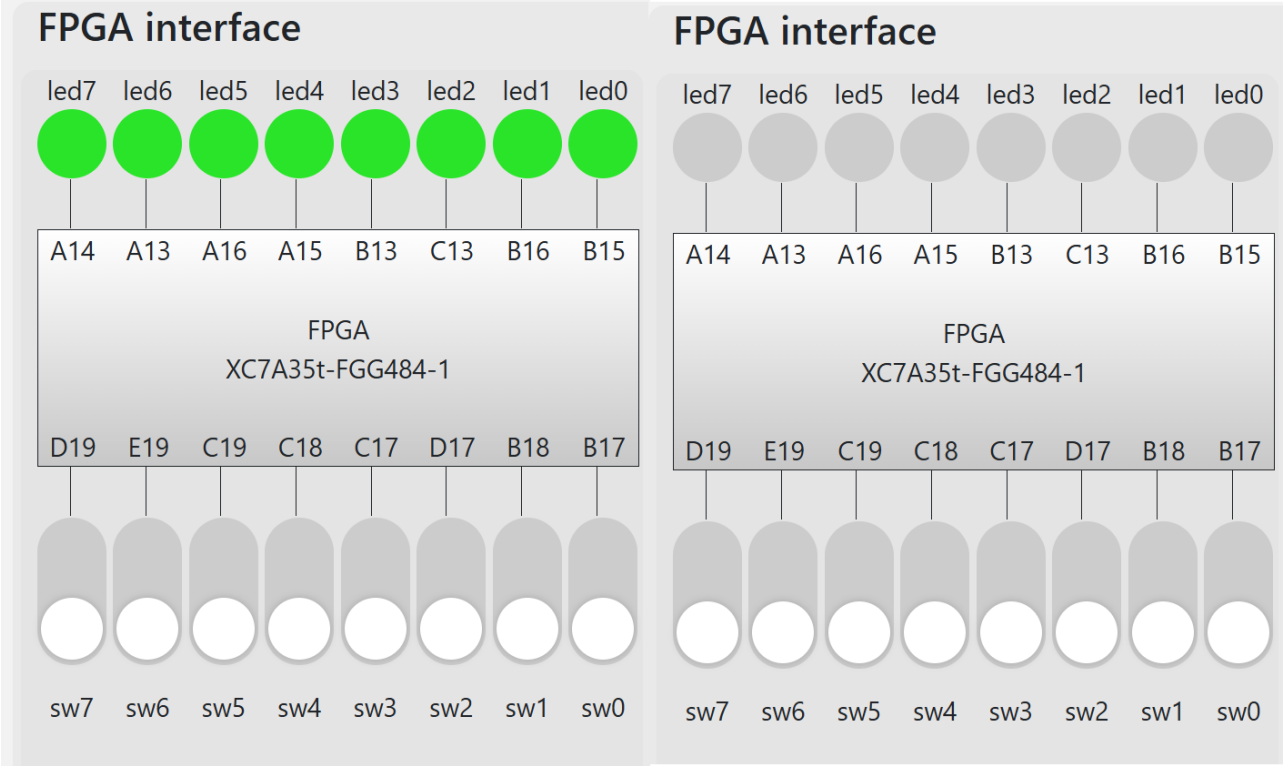


四、测试结果与分析

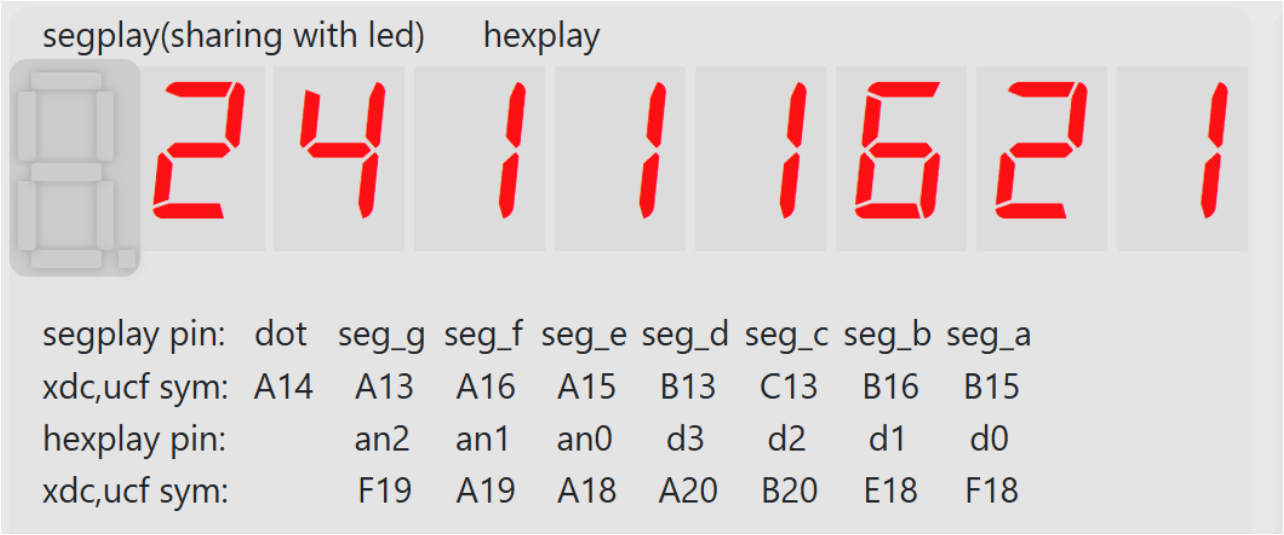
1. 必做题目1的 FPGAOL 运行结果如下，这里只是开了 7，4，3，2 四个开关，按照题目要求，应该亮的 LED 应为 4，7，0，1 ，这与FPGAOL 运行结果一致。



2. 必做题目2的 FPGAOL 运行结果如下，由于 pdf 文档中无法显示动图/视频，故只截取了一个灯亮的图片与一个灯灭的图片。



3. 必做题目3的 FPGAOL 运行结果如下，正确显示了我的学号 24111621 。



4. 选择性必做题目3的 FPGAOL 运行结果如下，理论上，在关了开关 sw5，sw3，sw1，sw0 后，从右往左数第 2，4，6 个数码管应该熄灭，3，5，7，8 号数码管仍然亮，但由题目要求，右边第一个数码管常

亮，所以在关了 sw0 后，右边第一个数码管还是显示对应数字。FPGAOL 运行结果与理论分析一致。



五、总结

通过本次实验，我初步了解了 FPGAOL 网站的使用，学会调整约束文件并生成比特流文件，让自己的代码能够在 FPGAOL 上运行，代码与对应的按钮开关联系起来了，对我来说这是一件很神奇的事情！其次我也了解了一些信号的处理方法，以及如何去应对常见的 Warning ，能够更快更准确地修正自己的代码。